Wink

Agent conformité — `compliance_agent`

EPIC-03 — Conformité & Normes réglementaires

Objet du document

Rôle actuel de l'agent, ses 3 tools, son mode de raisonnement, et son point d'intégration prévu avec le futur copilot.
À donner tel quel à une nouvelle session pour reprendre ce chantier sans perdre le contexte.

20 septembre 2026

Table des matières

1 Contexte et périmètre **2**

2 Ce que fait l'agent (pilote HealthTech, service 3.3.6) **2**
2.1 Avant / Maintenant 2
2.2 Un run typique 3

3 Les 3 tools **3**

4 Comment il raisonne, et garde-fous **3**
4.1 Règles imposées par le system prompt 4
4.2 Comment il choisit quel tool appeler 4

5 Pourquoi les tools ne sont pas dans `agents/` **4**

6 Rôle prévu dans le futur copilot — pas encore construit **5**

7 Fichiers touchés **5**

8 Marche à suivre pour tester **6**
8.1 Préparation 6
8.2 Test technique — ça tourne sans erreur ? 6
8.3 Validation de fond — est-ce correct sur le plan réglementaire ? 7

1 Contexte et périmètre

Wink prévoit 3 agents à terme, un par grand domaine métier (décision actée en conversation produit, pas encore reportée dans `Wink_architecture_fichiers.md`) :

— `administrative_agent` (EPIC-01, I1) — pas encore créé, pas de code.
— `compliance_agent` (EPIC-03) — celui documenté ici, renommé depuis `regulatory_agent`.
— `contract_agent` (EPIC-04) — dossier réservé (`agents/contract_agent/`), vide.

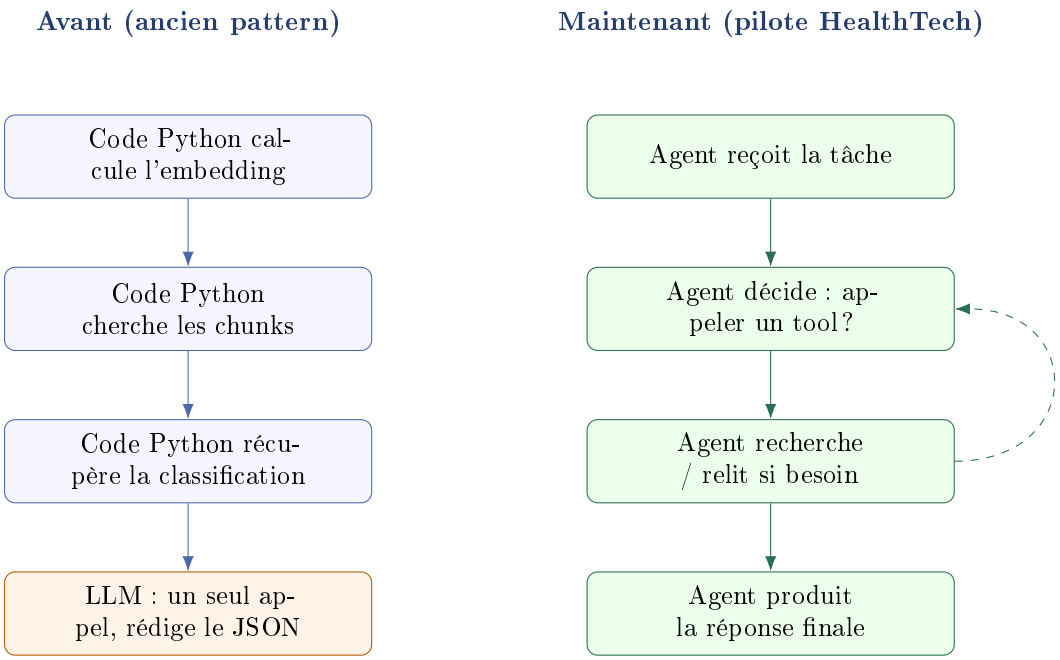
`compliance_agent` couvre aujourd'hui 5 tâches :

Service	Tâche
3.3.5 (étape 0)	Détection de secteur
3.3.1	Diagnostic RGPD
3.3.12	Classification AI Act
3.3.6	Diagnostic d'écart HealthTech
3.3.6	Diagnostic d'écart FinTech

Seule la tâche HealthTech gap-analysis a été convertie en agent à raisonnement autonome (tool-calling) — c'est le **pilote**. RGPD, AI Act, FinTech gap-analysis et détection de secteur restent sur l'ancien pattern (un seul appel LLM, RAG pré-effectué par le code appelant, pas de tools). Conversion prévue service par service, jamais en une fois.

2 Ce que fait l'agent (pilote HealthTech, service 3.3.6)

2.1 Avant / Maintenant



Le code Python ne décide plus rien à l'avance : il donne à l'agent le chapitre à traiter et une requête de recherche suggérée, puis le laisse **décider lui-même** s'il doit consulter le profil du projet, rechercher dans le corpus légal, combien de fois, avec quelle formulation.

2.2 Un run typique

1. L'agent reçoit la tâche (chapitre à traiter + requête suggérée).
2. Il appelle `lire_profil_projet` pour connaître la classe de risque MDR du projet.
3. Il appelle `rechercher_corpus_reglementaire` avec la requête suggérée (ou une reformulation).
4. Si les résultats lui semblent insuffisants, il peut rechercher à nouveau avec une autre requête.
5. Il produit la liste d'obligations, chacune citant le `source_id` réel d'un chunk qu'il a vu.

3 Les 3 tools

Tous en **lecture seule** — l'agent ne persiste jamais rien lui-même, ça reste le rôle du service backend appelant (`upsert_result`). Définis dans `backend/entrepreneur/services/compliance_agent_tools_service.py` (pas dans `agents/compliance_agent/`, voir section 5).

1. `rechercher_corpus_reglementaire(domaine, requete)`

Recherche sémantique dans le corpus légal interne déjà ingéré (`content_embeddings`), filtrée par domaine (`secteur_healthtech`, `secteur_fintech`, `rgpd`, `ai_act`...).
Renvoie : une liste de chunks `{source_id, texte}`.
En interne : `embed_texts` (embedding de la requête) puis `search_similar_chunks` (recherche cosinus, top 8).

2. `lire_profil_projet()`

Contexte réglementaire ET business déjà connu du projet : secteur détecté, classification (classe de risque MDR ou type d'agrément FinTech) si elle existe, réponses au questionnaire sectoriel, et champs business du projet (stage, description, technologies, customer_segments, target_users, value_proposition).
Sert à : contextualiser une génération.

3. `lire_diagnostics_existants()`

Résultats déjà persistés pour ce projet (obligations RGPD, classification AI Act, chapitres de gap-analysis HealthTech/FinTech avec leurs obligations et le statut manuel « accompli »).
Sert à : répondre sans régénérer — pensé pour le copilot (section 6), pas encore utilisé par le pilote actuel qui ne fait que générer.

Explicitement exclus

— Un **tool d'écriture** — l'agent ne modifie jamais la DB (frontière de sécurité, pas une limite technique).

— Un **tool de recherche web** — casserait la garantie de source vérifiable (voir section 4).

4 Comment il raisonne, et garde-fous

Implémenté avec **pydantic-ai** (`agents/compliance_agent/healthtech_gap_analysis_service.py`) : un `pydantic_ai.Agent` construit avec un modèle Groq (`openai/gpt-oss-120b`), un system prompt, les 3 tools, et un schéma de sortie structuré (`GapAnalysisResult`).

4.1 Règles imposées par le system prompt

— Jamais d'obligation inventée sans chunk réellement vu pendant la tâche en cours.
— Les obligations doivent tenir compte de la classe de risque trouvée via `lire_profil_projet`.
— Liste vide acceptée et correcte si le corpus ne couvre pas le sujet — **jamais de recherche web de secours**. Un trou dans le corpus doit remonter pour un enrichissement contrôlé, pas être patché silencieusement à chaque appel.
— Peut suggérer textuellement qu'une obligation semble déjà couverte par un fait du profil, mais ne coche jamais lui-même une case « accompli » — action manuelle de l'entrepreneur.
— `source_id` doit être un identifiant réel vu via le tool de recherche, jamais inventé.

Garde-fous d'exécution

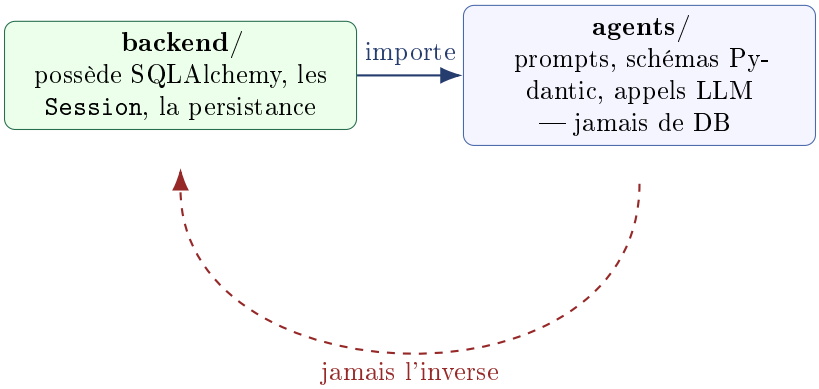
— `UsageLimits(request_limit=10, tool_calls_limit=6)` — borne le nombre d'appels pour éviter qu'un agent jamais satisfait de ses recherches ne boucle indéfiniment (chaque tour coûte un appel LLM réel).
— **Vérification a posteriori** (`_obligations_to_dict`, côté backend) : chaque `source_id` renvoyé par l'agent est comparé à l'ensemble des `source_id` réellement vus pendant ce run. Un `source_id` absent (hallucination) devient `None`, pas une erreur bloquante.

4.2 Comment il choisit quel tool appeler

C'est le LLM lui-même, pas du code. Le system prompt décrit ce que fait chaque tool (via sa signature et sa docstring, exposées automatiquement par pydantic-ai comme schéma de function-calling) ; à chaque étape, le modèle Groq décide s'il a besoin d'un tool, lequel, avec quels arguments, en se basant sur la tâche reçue et les résultats des appels précédents — le mécanisme de function-calling natif de Groq (API compatible OpenAI).

5 Pourquoi les tools ne sont pas dans `agents/`

Règle déjà actée avant ce chantier : `agents/` **ne touche jamais la DB directement** (`backend/` possède SQLAlchemy et les `Session`).



Deux des trois tools ont besoin d'une `Session` pour lire la DB — donc leur implémentation vit dans `backend/entrepreneur/services/compliance_agent_tools_service.py`, fermée par closure sur `(db, user, project_id)`, puis **injectée** à l'agent au moment de l'appel :

```
generate_gap_analysis(chapitre_titre, requete, tools)
```

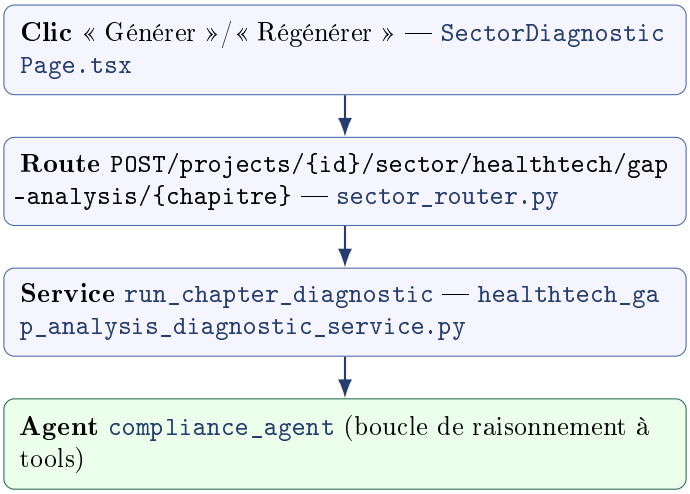
Cela respecte le sens de dépendance déjà en place sans dépendance circulaire. Le fichier `agents/compliance_agent/healthtech_gap_analysis_service.py` ne sait pas que ces fonctions touchent

une DB — il reçoit juste une liste de fonctions Python à enregistrer sur l'`Agent`. L'endroit où vit le code d'un tool ne change rien au fait que c'est bien le LLM qui décide de l'appeler : c'est une question d'organisation de dépôt, pas de définition d'un agent.

6 Rôle prévu dans le futur copilot — pas encore construit

État actuel — ce qui existe réellement

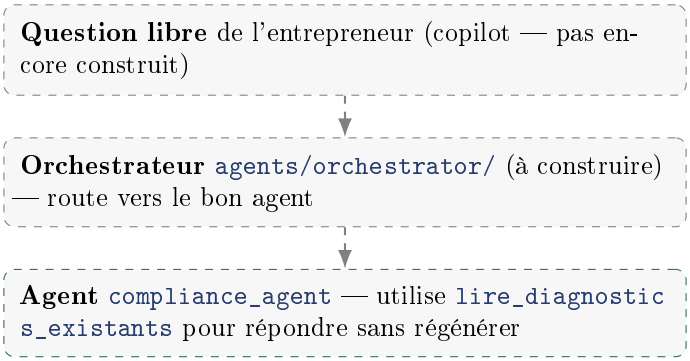
`compliance_agent` n'est appelé QUE depuis la chaîne suivante :



Aucun copilot n'existe encore — `agents/orchestrator/` est un dossier vide (juste un `.gitkeep`), aucune route de conversation libre n'est câblée.

Ce qui est prévu — décision produit, pas encore implémenté

Le futur copilot entrepreneur posera des questions libres (« est-ce que je dois faire une AIPD ? », « où en est mon dossier MDR ? »).



C'est précisément pour cet usage que `lire_diagnostics_existants` a été conçu dès maintenant, même si le pilote actuel (génération pure) ne s'en sert pas encore. Reste à construire : la route de conversation libre, l'orchestrateur qui décide vers quel agent router, et la conversion des 4 autres tâches (RGPD, AI Act, FinTech gap, détection secteur) au même pattern à tools que le pilote HealthTech.

7 Fichiers touchés

Fichier	Changement
agents/compliance_agent/	Renommé depuis regulatory_agent/ (5 fichiers : ai_act_service.py, fintech_gap_analysis_service.py, healthtech_gap_analysis_service.py [converti en agent], rgpd_service.py, sector_detection_service.py)
compliance_agent_tools_service.py	Nouveau fichier (backend/entrepreneur/services/) — les 3 tools, accès DB
healthtech_gap_analysis_diagnostic_service.py	Réécrit pour construire les tools et appeler l'agent au lieu de faire le RAG lui-même
llm_config.yaml / prompts.yaml	Clés renommées regulatory_agent_* → compliance_agent_* ; nouvelle clé system (remplace diagnostic) pour compliance_agent_healthtech_gap
backend/requirements.txt	Ajout pydantic-ai-slim[groq]>=2.46.0, bump pydantic>=2.12.0 et groq>=0.25.0
RegulatoryAgentError	Renommé RgpdAgentError (cohérence avec les autres exceptions du dossier)

Non testé de bout en bout

GROQ_API_KEY existe déjà dans backend/.env (ajoutée pour l'extraction de documents, agents/shared/document_processing/) et sera réutilisée telle quelle par GroqProvider — mais aucun vrai appel Groq avec la boucle de tool-calling n'a encore été exécuté sur ce chantier : seule la syntaxe et la résolution des imports ont été vérifiées (py_compile).

8 Marche à suivre pour tester

8.1 Préparation

1. GROQ_API_KEY est déjà présente dans backend/.env — rien à faire.
2. Installer les nouvelles dépendances dans le **vrai** environnement du projet (.venv Windows) : `pip install -r backend/requirements.txt`.

8.2 Test technique — ça tourne sans erreur ?

Le plus rapide : lancer le backend (uvicorn), ouvrir un projet HealthTech au questionnaire complet, cliquer « Générer » sur le premier chapitre, regarder les logs serveur.

Pour un test isolé sans passer par l'UI :

```
from backend.shared.database import SessionLocal
from backend.entrepreneur.services.compliance_agent_tools_service import
build_compliance_agent_tools
from agents.compliance_agent.healthtech_gap_analysis_service import
generate_gap_analysis

db = SessionLocal()
seen = []
tools = build_compliance_agent_tools(db, user, project_id, seen)
result = generate_gap_analysis("Obligations du fabricant", "obligations du
```

```
fabricant avant mise sur le marché MDR", tools)
print(result.model_dump_json(indent=2))
print("Chunks vus :", seen)
```

8.3 Validation de fond — est-ce correct sur le plan réglementaire ?

— **Traçabilité** : logue systématiquement les `seen_source_ids` à côté du résultat — vérifie que le contenu cité est pertinent, pas juste que l'ID existe.

— **Relecture humaine** : pour chaque `source_id` cité, relire le chunk réel (ou l'article MDR correspondant sur EUR-Lex) et vérifier que l'obligation générée en est une interprétation fidèle. Aucun test automatisé ne remplace ça pour les premiers runs de chaque chapitre.

— **Sensibilité à la classe de risque** : générer le même chapitre pour deux projets de classes différentes (I vs III) — les obligations doivent différer en conséquence.

— **Cas limites** : classe de risque non déterminée (ne doit rien halluciner), domaine sans chunk pertinent (doit renvoyer une liste vide justifiée), rejouer deux fois le même chapitre à température 0 (résultats globalement stables).

— **Consommation** : `result.usage()` donne le nombre de requêtes/tool-calls réellement utilisés — doit rester loin des limites (6 tool-calls, 10 requêtes) ; sinon, l'agent cherche trop ou le corpus est mal indexé pour ce chapitre.